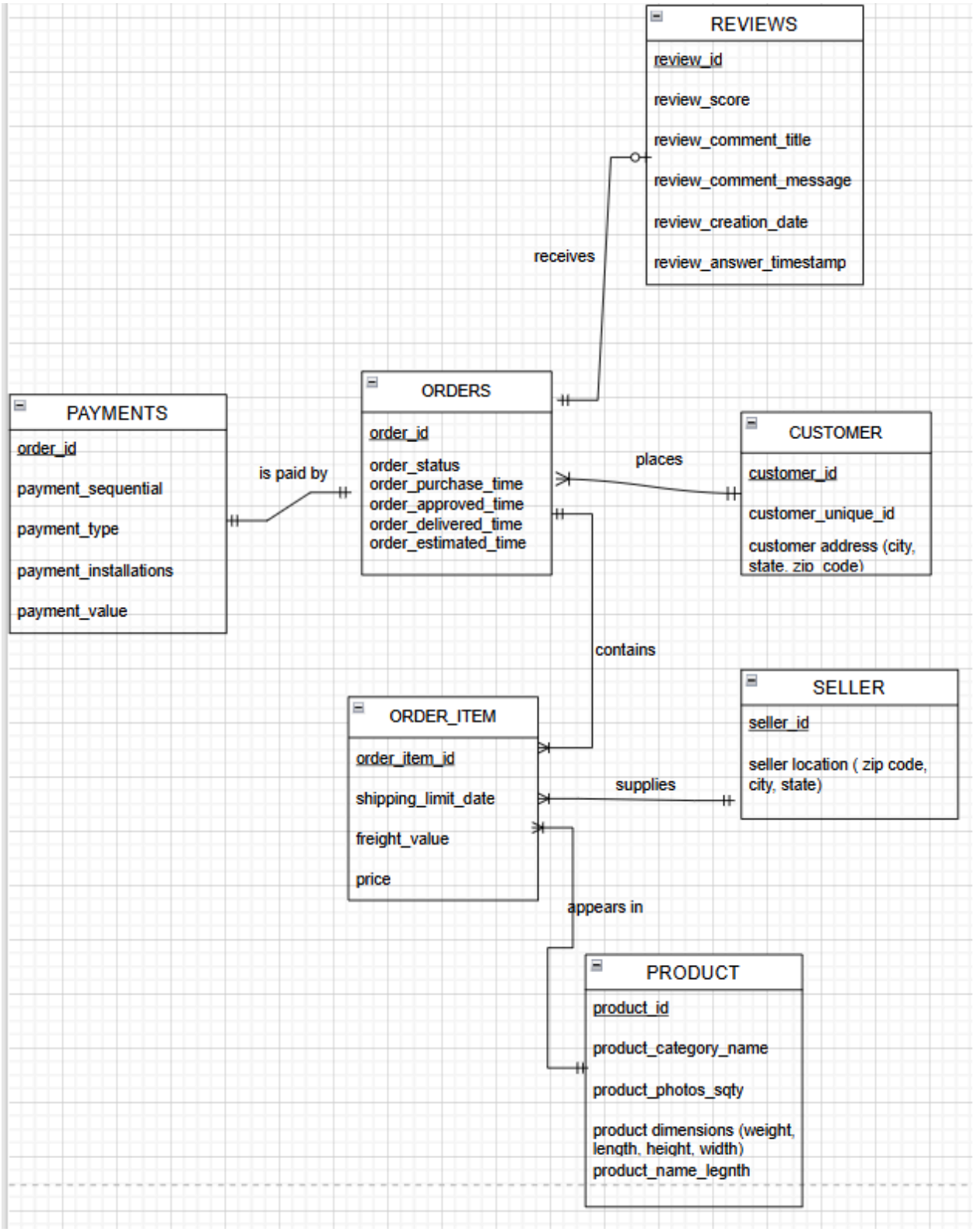
First ER Diagram



Metadata

CUSTOMERS

Variable name	Definition	Allowable Values	Examples
Customer_id (PK)	Unique identifier for each customer record, tied to a specific order	32-character alphanumeric string (UUID), non-null, unique	06b8999e2fba1a1fbc88172c00ba8bc7
Customer_unique_id	Unique identifier for the actual customer	32-character alphanumeric string (UUID), non-null	861eff4711a542e4b93843c6dd7febb0
customer_zip_code_prefix	5-digit zip code prefix of the customer's delivery address	Integer (5 digits), non-null	14409, 09790, 01151
customer_city	City of the customer's delivery address	String (lowercase), non-null	sao paulo, franca, sao bernardo do campo
customer_state	Brazilian state abbreviation of the customer's delivery address	2-character Brazilian state code, non-null	SP, RJ, MG, RS, PR

ORDERS

Variable name	Definition	Allowable Values	Examples
order_id (PK)	Unique identifier for each order	32-character alphanumeric string (UUID), non-null, unique	e481f51cbdc54678b7cc49136f2d6af7
customer_id (FK)	Foreign key linking the order to the customer who placed it	32-character alphanumeric string (UUID), non-null	9ef432eb6251297304e76186b10a928d
order_status	Current fulfillment status of the order in its lifecycle	delivered, shipped, canceled, invoiced, processing, created, approved, unavailable	delivered, shipped
order_purchase_timestamp	Date and time when the customer placed the order	Datetime (YYYY-MM-DD HH:MM:SS), non-null	2017-10-02 10:56:33

order_approved_at	Date and time when the order payment was approved	Datetime (YYYY-MM-DD HH:MM:SS), nullable	2017-10-02 11:07:15
order_delivered_carrier_date	Date and time when the order was handed off to the logistics carrier	Datetime (YYYY-MM-DD HH:MM:SS), nullable	2017-10-04 19:55:00
order_delivered_customer_date	Date and time when the order was delivered to the customer	Datetime (YYYY-MM-DD HH:MM:SS), nullable	2017-10-10 21:25:13
order_estimated_delivery_date	Estimated delivery date provided to the customer at time of purchase	Datetime (YYYY-MM-DD HH:MM:SS), non-null	2017-10-18 00:00:00

SELLERS

Variable name	Definition	Allowable Values	Examples
seller_id(PK)	Unique identifier for each seller on the platform	32-character alphanumeric string (UUID), non-null, unique	3442f8959a84dea7ee197c632cb2df15
seller_zip_code_prefix	5-digit zip code prefix of the seller's registered business address	Integer (5 digits), non-null	13023, 13844, 20031, 04195
seller_city	City of the seller's registered business address	String (lowercase), non-null	sao paulo, campinas, mogi guacu, rio de janeiro
seller_state	Brazilian state abbreviation of the seller's registered business address	2-character Brazilian state code, non-null	SP, PR, RJ, MG

ORDER_ITEMS

Variable name	Definition	Allowable Values	Examples
order_id (PK) (FK)	Foreign key linking the item to its parent order. Forms part of the composite	32-character alphanumeric string (UUID), non-null	00010242fe8c5a6d1ba2dd792cb16214

	primary key with order_item_id		
order_item_id (PK) (FK)	Sequential number identifying each item within a given order. Resets to 1 for each new order. Forms part of the composite primary key with order_id	Integer, non-null	1, 2, 3 ... 21
product_id (FK)	Foreign key linking the item to its product	32-character alphanumeric string (UUID), non-null	4244733e06e7ecb4970a6e2683c13e61
seller_id (FK)	Foreign key identifying which seller fulfilled this item	32-character alphanumeric string (UUID), non-null	48436dade18ac8b2bce089ec2a041202
shipping_limit_date	Deadline by which the seller must hand the item to the carrier	Datetime (YYYY-MM-DD HH:MM:SS), non-null	2017-09-19 09:45:35
price	Listed sale price of the individual item in Brazilian Reais	Decimal, non-null	58.90, 239.90, 199.00
freight_value	Shipping cost charged for this item in Brazilian Reais	Decimal, non-null	13.29, 19.93, 17.87

PAYMENTS

Variable name	**Definition**	**Allowable Values**	**Examples**
order_id (PK, FK)	Foreign key linking the payment to its parent order. Forms part of the composite primary key with payment_sequential	32-character alphanumeric string (UUID), non-null	b81ef226f3fe1789b1e8b2acac839d17
Payment_sequential (PK)	Sequential number identifying each payment method used within a given order. Allows a single order to be split	Integer, non-null	1, 2, 3 ... 29

	across multiple payment types		
payment_type	Method used to make the payment	credit_card, UPI, boleto, voucher, debit_card, not_defined	credit_card, UPI
payment_installments	Number of installments the customer chose to split the payment into	Integer, non-null	1, 8, 24
payment_value	Total amount paid via this payment method in Brazilian Reais	Decimal, non-null	99.33, 24.39, 65.71

ORDER_REVIEWS

Variable name	Definition	Allowable Values	Examples
review_id (PK)	Unique identifier for each review	32-character alphanumeric string (UUID), non-null, unique	7bc2406110b926393aa56f80a40eba40
order_id (FK)	Foreign key linking the review to its parent order	32-character alphanumeric string (UUID), non-null	73fc7af87114b39712e6da79b0a377eb
review_score	Star rating given by the customer for the order	Integer 1–5, non-null	1, 2, 3, 4, 5
review_comment_title	Title of the review	String, nullable	"I recommend", "Perfect delivery"
review_creation date	Date the review survey was sent to the customer	Datetime (DD/MM/YY HH:MM), non-null	18/01/18 0:00, 17/02/18 0:00
review_answer_timestamp	Date and time when the customer submitted their review response	Datetime (DD/MM/YY HH:MM), non-null	18/01/18 21:46, 11/03/18 3:05

PRODUCTS

Variable name	Definition	Allowable Values	Examples

product_id(PK)	Unique identifier for each product listed on the platform	32-character alphanumeric string (UUID), non-null, unique	1e9e8ef04dbcff4541ed26657ea517e5
product_category	Category the product belongs to	String (lowercase), nullable	perfumery, Art, sport leisure
product_name_length	Number of characters in the product name	Integer, nullable	40, 44, 46
product_description_length	Number of characters in the product description	Integer, nullable	287, 276, 250
product_photos_qty	Number of photos uploaded for the product listing	Integer, nullable	1, 3, 5
product_weight_g	Weight of the product in grams	Integer (grams), nullable	225, 1000, 154
product_length_cm	Length of the product packaging in centimetres	Integer (cm), nullable	16, 30, 18
product_height_cm	Height of the product packaging in centimetres	Integer (cm), nullable	10, 18, 9
product_width_cm	Width of the product packaging in centimetres	Integer (cm), nullable	14, 20, 15

Organization Rules

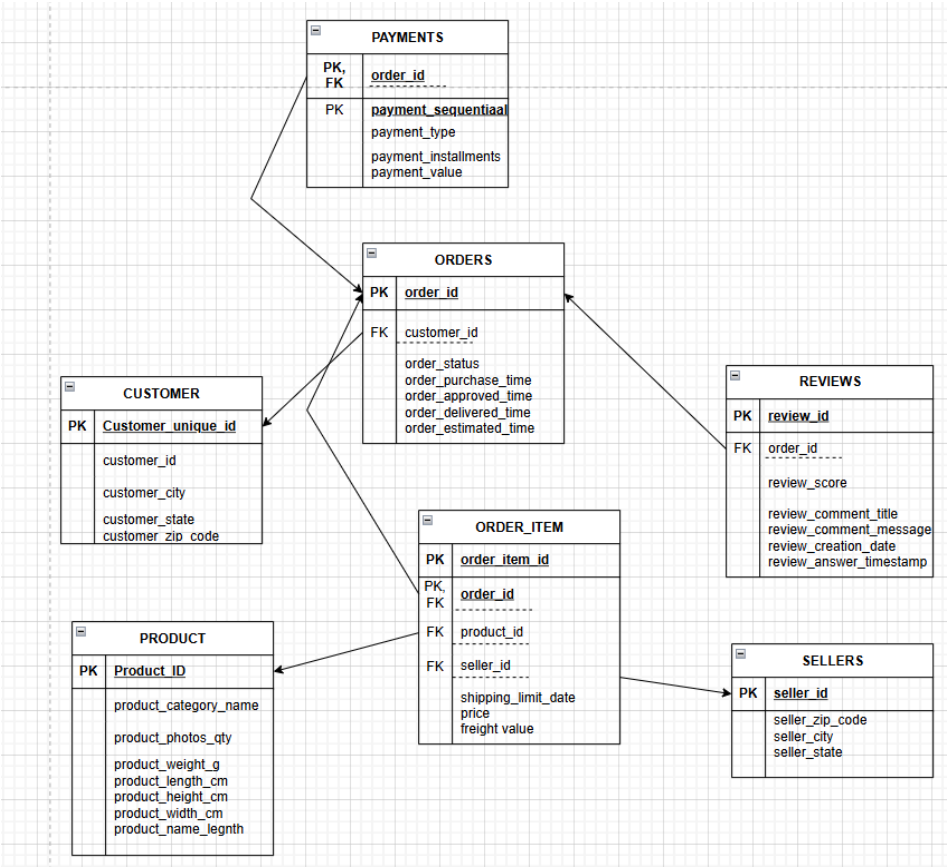
1. Order must belong to exactly one customer.
2. An order must have at least one order_item.
3. Line items must reference a valid product and seller.
4. Payments must reference an existing order; payment_value must be ≥ 0.
    - If split tender is allowed, (order_id, payment_sequential) must be unique.
5. Review score must be integer 1–5; at most one review per order (recommended).
    - Enforce unique constraint on reviews.order_id (or business logic).
6. Deletion policy (typical analytics-safe approach):
    - Do not hard-delete orders/payments/order_items/products; use status changes (e.g., canceled) to preserve auditability.
    - If a customer must be removed (privacy), anonymize customer_unique_id rather than deleting transactional history.

Tasks in Stage 2

- Design a Logical Database:
  1) Find data for at least two entities.
  2) Draw the graphical relational schema based on your tables.
  3) Draw the as-is dependency diagram and identify types of dependencies.
  4) Do you have tables in 3rd normal form? If not, convert your tables to 3rd normal form.

Part 2 NOTES: Professor said we should focus on data relations and specific entities that could (hypothetically if not included) be logged to give a competitive edge on competitors (walmart, costco etc). This would be nice to include in a slide or at some point in the project. Could be worthwhile to discuss the attribute "product_photos_qty" in product, and see how this affects sales

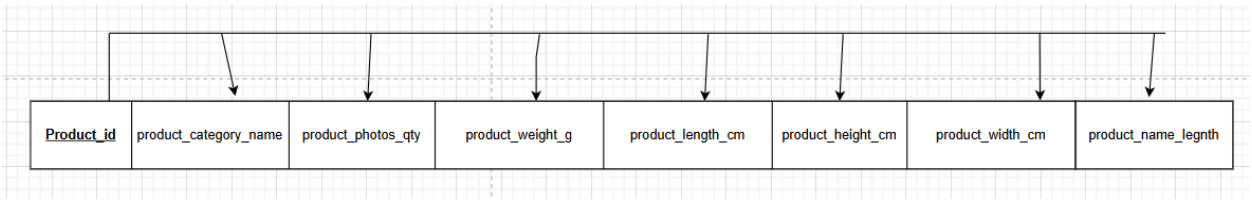
Relational Data Model



As- Is Dependencies

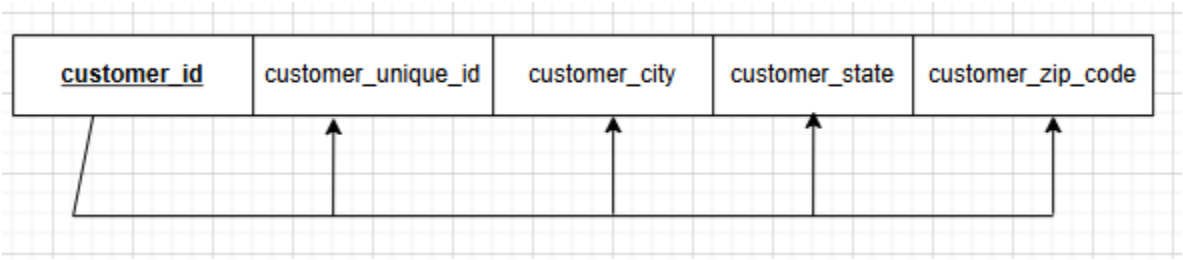
PRODUCTS

Already in 3NF



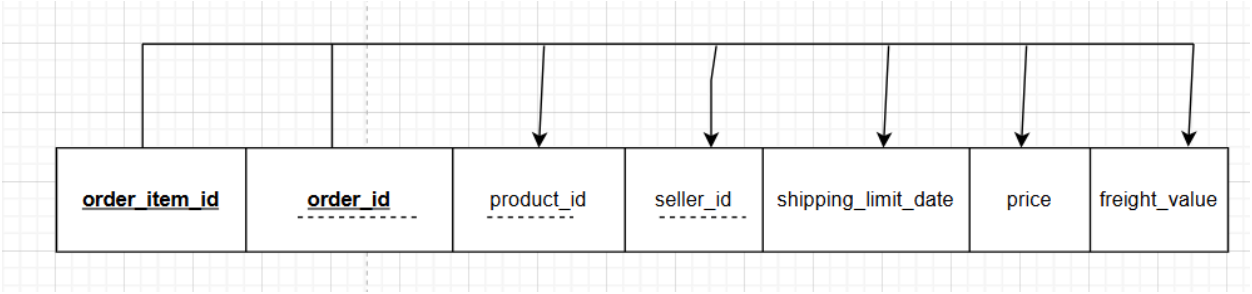
CUSTOMERS

This table is already in third normal form. Some zip codes can be in multiple states. Some cities have the same names in multiple states. Therefore, we can't treat as transitive



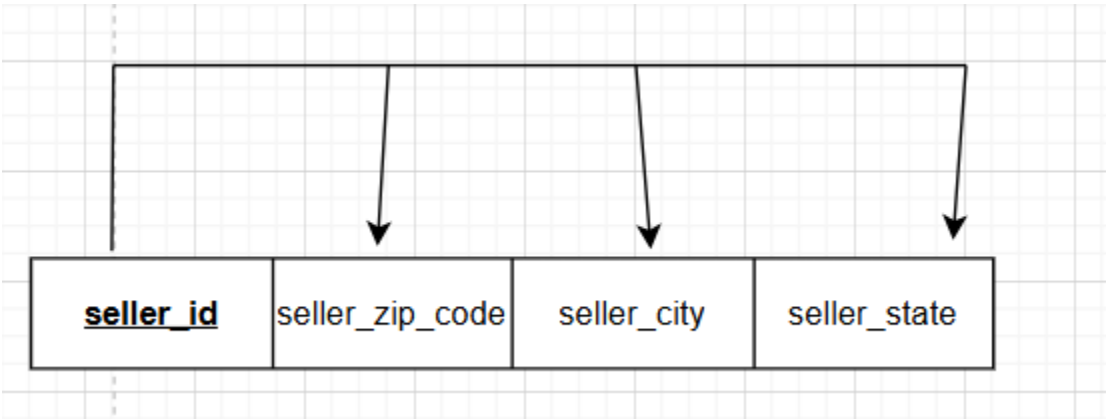
ORDER_ITEM

This table is already in third normal form



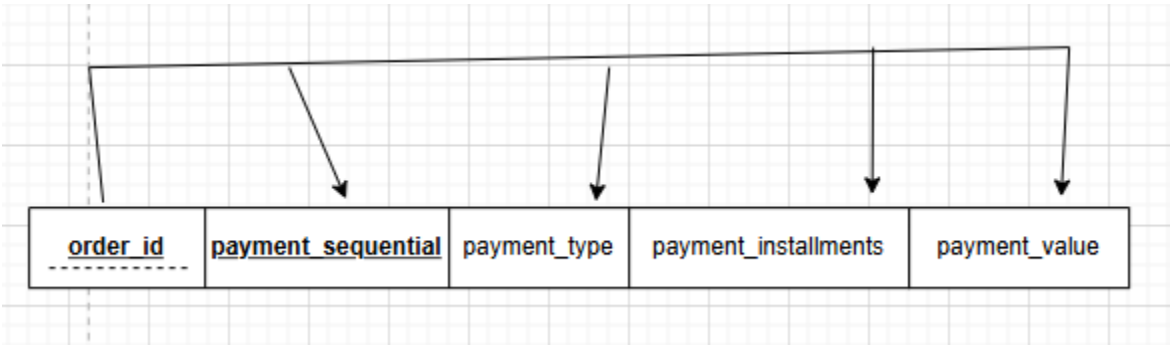
SELLERS

This table is already in third normal form

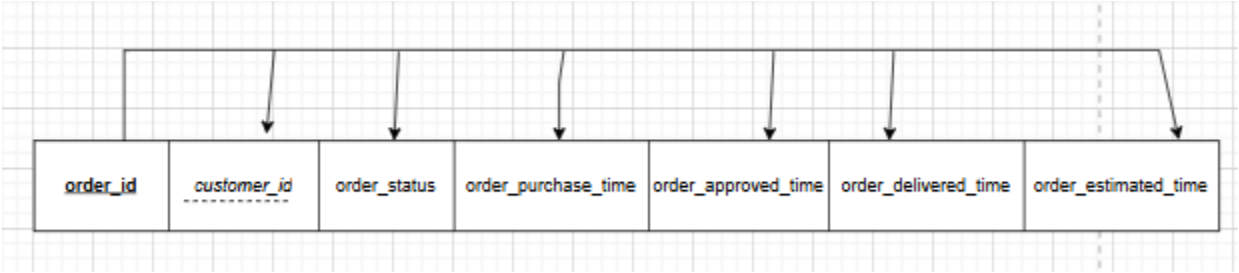


PAYMENTS

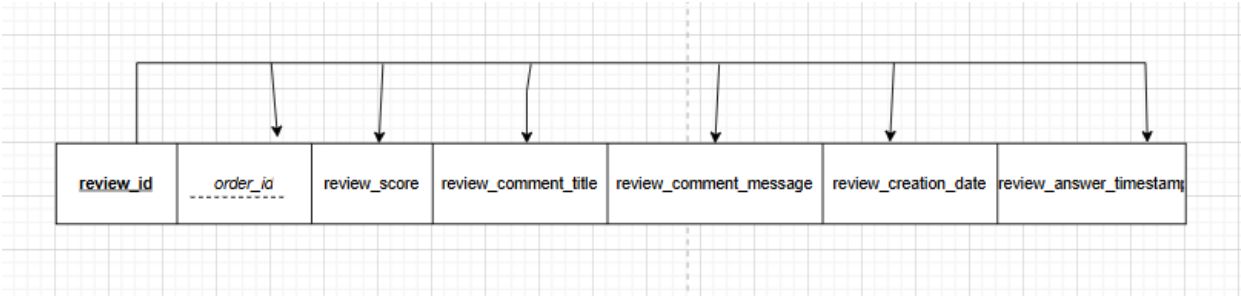
This table is already in 3NF



ORDERS



REVIEWS



ACTUAL DATA TABLES IN 3NF

```
325 •   SELECT * FROM customers;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
00012a2ce6f8dcda20d059ce98491703	248ffe10d632bebe4f7267f1f44844c9	6273	osasco	SP
000161a058600d5901f007fab4c27140	b0015e09bb4b6e47c52844fab5fb6638	35550	itapecerica	MG
0001fd6190edaaf884bcaf3d49edf079	94b11d37cd61cb2994a194d11f89682b	29830	nova venecia	ES
0002414f95344307404f0ace7a26f1d5	4893ad4ea28b2c5b3ddf4e82e79db9e6	39664	mendonca	MG
000379cdec625522490c315e70c7a9fb	0b83f73b19c2019e182fd552c048a22c	4841	sao paulo	SP
0004164d20a9e969af783496f3408652	104bdb7e6a6cdceaa88c3ea5fa6b2b93	13272	valinhos	SP
000419c5494106c306a97b5635748086	14843983d4a159080f6afe4b7f346e7c	24220	niteroi	RJ
00046a560d407e99b969756e0b10f282	0b5295fc9819d831f68eb0e9a3e13ab7	20540	rio de janeiro	RJ
00050bf6e01e69d5c0fd612f1bcfb69c	e3cf594a99e810f58af53ed4820f25e5	98700	ijui	RS
000598caf2ef4117407665ac33275130	7e0516b486e92ed3f3afdd6d1276cfbd	35540	oliveira	MG
0005aefbb696d34b3424dccd0a0e9fd0	616309b2eeb7bd9c05b0fdfbab28e6c6	3052	sao paulo	SP

```
327 •   SELECT * FROM geolocation;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
1037	-23.54562128	-46.63929205	sao paulo	SP
1046	-23.54608113	-46.64482030	sao paulo	SP
1046	-23.54612897	-46.64295148	sao paulo	SP
1041	-23.54439216	-46.63949931	sao paulo	SP
1035	-23.54157796	-46.64160722	sao paulo	SP
1012	-23.54776230	-46.63536054	são paulo	SP
1047	-23.54627311	-46.64122517	sao paulo	SP
1013	-23.54692321	-46.63426370	sao paulo	SP
1029	-23.54376906	-46.63427784	sao paulo	SP
1011	-23.54763955	-46.63603162	sao paulo	SP
1013	-23.54732513	-46.63418379	sao paulo	SP

```
328 •   SELECT * FROM sellers;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

seller_id	seller_zip_code_prefix	seller_city	seller_state
0015a82c2db000af6aaaf3ae2ecb0532	9080	santo andre	SP
001cca7ae9ae17fb1caed9dfb1094831	29156	cariacica	ES
001e6ad469a905060d959994f1b41e4f	24754	sao goncalo	RJ
002100f778ceb8431b7a1020ff7ab48f	14405	franca	SP
003554e2dce176b5555353e4f3555ac8	74565	goiania	GO
004c9cd9d87a3c30c522c48c4fc07416	14940	ibitinga	SP
00720abe85ba0859807595bbf045a33b	7070	guarulhos	SP
00ab3eff1b5192e5f1a63bcecfee11c8	4164	sao paulo	SP
00d8b143d12632bad99c0ad66ad52825	30170	belo horizonte	MG
00ee68308b45bc5e2660cd833c3f81cc	3333	sao paulo	SP
00fc707aaaad2d31347cf883cd2dfe10	87025	maringa	PR

```
330 •   SELECT * FROM products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

product_id	product category	product_name_length	product_description_length	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm
00066f42ae...	perfumery	53	596	6	300.00	20.00	16.00	16.00
00088930e...	automotive	56	752	4	1225.00	55.00	10.00	26.00
0009406fd7...	bed table ...	50	266	2	300.00	45.00	15.00	35.00
000b8f95fc...	housewares	25	364	3	550.00	19.00	24.00	12.00
000d9be29...	Watches ...	48	613	4	250.00	22.00	11.00	15.00
0011c512e...	automotive	58	177	1	100.00	16.00	15.00	16.00
00126f27c8...	Cool Stuff	42	2461	1	700.00	25.00	5.00	15.00
001795ec6f...	Games co...	53	274	1	600.00	30.00	20.00	20.00
001b237c0...	bed table ...	42	253	1	6000.00	40.00	4.00	30.00
001b72dfd6...	Furniture ...	45	520	3	600.00	26.00	8.00	22.00

```
332 •   SELECT * FROM orders;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
00010242f...	3ce436f18...	delivered	2017-09-13 08:59:02	2017-09-13 09:45:35	2017-09-19 18:34:16	2017-09-20 23:43:48	2017-09-29 00:00:00
00018f77f...	f6dd3ec06...	delivered	2017-04-26 10:53:06	2017-04-26 11:05:13	2017-05-04 14:35:00	2017-05-12 16:04:24	2017-05-15 00:00:00
000229ec3...	6489ae5e4...	delivered	2018-01-14 14:33:31	2018-01-14 14:48:30	2018-01-16 12:36:48	2018-01-22 13:19:16	2018-02-05 00:00:00
00024acbc...	d4eb9395c...	delivered	2018-08-08 10:00:35	2018-08-08 10:10:18	2018-08-10 13:28:00	2018-08-14 13:32:39	2018-08-20 00:00:00
00042b26c...	58dbd0b2d...	delivered	2017-02-04 13:57:51	2017-02-04 14:10:13	2017-02-16 09:46:09	2017-03-01 16:42:31	2017-03-17 00:00:00
00048cc3a...	816cbea96...	delivered	2017-05-15 21:42:34	2017-05-17 03:55:27	2017-05-17 11:05:55	2017-05-22 13:44:35	2017-06-06 00:00:00
00054e843...	32e2e6ab0...	delivered	2017-12-10 11:53:48	2017-12-10 12:10:31	2017-12-12 01:07:48	2017-12-18 22:03:38	2018-01-04 00:00:00
000576fe3...	9ed5e522d...	delivered	2018-07-04 12:08:27	2018-07-05 16:35:48	2018-07-05 12:15:00	2018-07-09 14:04:07	2018-07-25 00:00:00
0005a1a17...	16150771d...	delivered	2018-03-19 18:40:33	2018-03-20 18:35:21	2018-03-28 00:37:42	2018-03-29 18:17:31	2018-03-29 00:00:00
0005f5044...	351d3cb2c...	delivered	2018-07-02 13:59:39	2018-07-02 14:10:56	2018-07-03 14:25:00	2018-07-04 17:28:31	2018-07-23 00:00:00

```
333 •  SELECT * FROM order_items;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.90	13.29
00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-05-03 11:05:13	239.90	19.93
000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-01-18 14:48:30	199.00	17.87
00024acbcdf0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-08-15 10:10:18	12.99	12.79
00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-02-13 13:57:51	199.90	18.14
00048cc3ae777c65dbb7d2a0634bc1ea	1	ef92defde845ab8450f9d70c526ef70f	6426d21aca402a131fc0a5d0960a3c90	2017-05-23 03:55:27	21.90	12.69
00054e8431b9d7675808bcb819fb4a32	1	8d4f2bb7e93e6710a28f34fa83ee7d28	7040e82f899a04d1b434b795a43b4617	2017-12-14 12:10:31	19.90	11.85
000576fe39319847cbb9d288c5617fa6	1	557d850972a7d6f792fd18ae1400d9b6	5996cddab893a4652a15592fb58ab8db	2018-07-10 12:30:45	810.00	70.75
0005a1a1728c9d785b8e2b08b904576c	1	310ae3c140ff94b03219ad0adc3c778f	a416b6a846a11724393025641d4edd5e	2018-03-26 18:31:29	145.95	11.65
0005f50442cb953dcd1d21e1fb923495	1	4535b0e1091c278dfd193e5a1d63b39f	ba143b05f0110f0dc71ad71b4466ce92	2018-07-06 14:10:56	53.99	11.40
00061f2a7bc09da83e415a52dc8a4af1	1	d63c1011f49d98b976c352955b1c4bea	cc419e0650a3c5ba77189a1882b7556a	2018-03-29 22:28:09	59.99	8.88

```
335 •  SELECT * FROM payments;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	payment_sequential	payment_type	payment_installments	payment_value
00010242fe8c5a6d1ba2dd792cb16214	1	credit_card	2	72.19
00018f77f2f0320c557190d7a144bdd3	1	credit_card	3	259.83
000229ec398224ef6ca0657da4fc703e	1	credit_card	5	216.87
00024acbcdf0a6daa1e931b038114c75	1	credit_card	2	25.78
00042b26cf59d7ce69dfabb4e55b4fd9	1	credit_card	3	218.04
00048cc3ae777c65dbb7d2a0634bc1ea	1	UPI	1	34.59
00054e8431b9d7675808bcb819fb4a32	1	credit_card	1	31.75
000576fe39319847cbb9d288c5617fa6	1	credit_card	10	880.75
0005a1a1728c9d785b8e2b08b904576c	1	credit_card	3	157.60
0005f50442cb953dcd1d21e1fb923495	1	credit_card	1	65.39
00061f2a7bc09da83e415a52dc8a4af1	1	credit_card	3	68.87

```
331 •  SELECT * FROM order_reviews;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

review_id	order_id	review_score	review_comment_title	review_creation_date	review_answer_timestamp
0001239bc1de2e33cb583967c2ca4c67	fc046d7776171871436844218f817d7d	5		20/03/18 0:00	20/03/18 18:36
0001cc6860aeaf5b9017fe4131a52e62	d4665434b01caa9dc3e3e78b3eb3593e	5		22/06/18 0:00	26/06/18 13:51
00020c7512a52e92212f12d3e37513c0	e28abf2eb2f1fbcbdc2dd0cd9a561671	5	Delivery RÃ i Pida	25/04/18 0:00	26/04/18 14:55
00032b0141443497c898b3093690af51	04fb47576993a3cb0c12d4b25eab6e4e	5		30/05/17 0:00	01/06/17 23:28
00034d88989f9a4c393bdcaec301537f	5f358d797a49fe2f24352f73426215f6	5		12/08/17 0:00	13/08/17 19:56
000359bceee1b6fe876b30d35b5f9ef2	97868e4c884bc85f4501ae0a66181cc9	5		06/02/18 0:00	06/02/18 20:54
00041b1f51e77a663df6db3f539d694f	998b1beee068744ac833dd8b31944623	1		24/05/18 0:00	24/05/18 12:45
00046a69550325aea5fb89f65c7387f2	9fbda7367628952bc36c3512c46d887b	5	Super recommend	01/07/18 0:00	02/07/18 12:32
0005534973388c830bb858cfba83b17b	a589caa6892ceacc6bbf2f8cc30a8ad4	5		01/09/17 0:00	02/09/17 13:04
00055e36e9608fe969231e551983a69c	f5fea26ab547eec920e6f8ecdc5c37e4	5		25/11/17 0:00	27/11/17 11:37
0005949d4c047d64863a6874338139ba	640883769bc5da4a077725646dc9995a	5		20/06/17 0:00	22/06/17 23:19

QUERIES

Query 1 — Monthly sales trend

Purpose: Find the strongest sales months.

```
SELECT
   DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS yearmonth,
   COUNT(DISTINCT o.order_id) AS total_orders,
   ROUND(SUM(oi.price), 2) AS total_revenue
```

```
FROM orders o
JOIN order_items oi
   ON o.order_id = oi.order_id
GROUP BY DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m')
ORDER BY total_revenue DESC;
```

Top revenue months include:
2017-11: about 1,010,271.37
2018-04: about 996,647.75
2018-05: about 996,517.68

Managerial importance
Helps Target prepare inventory, staffing, marketing, and logistics around seasonal peaks.

```
SELECT
    DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS yearmonth,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(SUM(oi.price), 2) AS total_revenue
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m')
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

yearmonth	total_orders	total_revenue
2017-11	7451	1010271.37
2018-04	6934	996647.75
2018-05	6853	996517.68
2018-03	7188	983213.44
2018-01	7220	950030.36
2018-07	6273	895507.22
2018-06	6160	865124.31
2018-08	6452	854686.33
2018-02	6694	844178.71
2017-12	5624	743914.17
2017-10	4568	664219.43
2017-09	4243	624401.69
2017-08	4293	573971.68
2017-05	3660	506071.14
2017-07	3969	498031.48
2017-06	3217	433038.60
2017-03	2641	374344.30

Query 2 — Top customer states by revenue
Purpose: Identify strongest geographic markets.
Uses JOIN

```
SELECT
   c.customer_state,
   COUNT(DISTINCT o.order_id) AS total_orders,
   ROUND(SUM(oi.price), 2) AS total_revenue
```

FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY c.customer_state
ORDER BY total_revenue DESC
LIMIT 10;
Finding

Top states by revenue:
SP: about 5.20M
RJ: about 1.82M
MG: about 1.59M

Managerial importance
Shows where demand is strongest and where Target should focus localized promotions and fulfillment capacity.

```sql
SELECT
    c.customer_state,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(SUM(oi.price), 2) AS total_revenue
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY c.customer_state
ORDER BY total_revenue DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

customer_state	total_orders	total_revenue
SP	41375	5202955.05
RJ	12762	1824092.67
MG	11544	1585308.03
RS	5432	750304.02
PR	4998	683083.76
SC	3612	520553.34
BA	3358	511349.99
DF	2125	302603.94
GO	2007	294591.95
ES	2025	275037.31

Query 3 — Top product categories by revenue
Purpose: Understand which categories drive business.
Uses JOIN

SELECT
    p.`product category` AS product_category,
    COUNT(*) AS total_order_lines,

```
    ROUND(SUM(oi.price), 2) AS total_revenue
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p.`product category`
ORDER BY total_revenue DESC
LIMIT 10;
```

Finding

Top categories include:
HEALTH BEAUTY: about 1.26M
Watches present: about 1.21M
bed table bath: about 1.04M

Managerial importance
Useful for assortment planning, pricing strategy, supplier negotiations, and ad spending.

```
198 •  SELECT
199        p.`product category` AS product_category,
200        COUNT(*) AS total_order_lines,
201        ROUND(SUM(oi.price), 2) AS total_revenue
202    FROM order_items oi
203    JOIN products p
204        ON oi.product_id = p.product_id
205    GROUP BY p.`product category`
206    ORDER BY total_revenue DESC
207    LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cel

product_category	total_order_lines	total_revenue
HEALTH BEAUTY	9670	1258681.34
Watches present	5991	1205005.68
bed table bath	11115	1036988.68
sport leisure	8641	988048.97
computer accessories	7827	911954.32
Furniture Decoration	8334	729762.49
Cool Stuff	3796	635290.85
housewares	6964	632248.66
automotive	4235	592720.11
Garden tools	4347	485256.46

Query 4 — Payment method mix
Purpose: Understand customer payment behavior.

```
SELECT
    payment_type,
    COUNT(*) AS payment_records,
```

```
    ROUND(SUM(payment_value), 2) AS total_payment_value,
    ROUND(AVG(payment_installments), 2) AS avg_installments
FROM payments
GROUP BY payment_type
ORDER BY total_payment_value DESC;
```

Finding
credit_card dominates with about 12.54M
UPI is second with about 2.87M
voucher and debit card are much smaller

Managerial importance
Supports checkout optimization, installment promotions, and payment-partner strategy.

```
SELECT
    payment_type,
    COUNT(*) AS payment_records,
    ROUND(SUM(payment_value), 2) AS total_payment_value,
    ROUND(AVG(payment_installments), 2) AS avg_installments
FROM payments
GROUP BY payment_type
ORDER BY total_payment_value DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_type	payment_records	total_payment_value	avg_installments
credit_card	76795	12542084.19	3.51
UPI	19784	2869361.27	1.00
voucher	5775	379436.87	1.00
debit_card	1529	217989.79	1.00
not_defined	3	0.00	1.00

Query 5 — Top sellers by revenue
Purpose: Identify best-performing sellers.
Uses JOIN

```
SELECT
    s.seller_id,
```

```
    s.seller_city,
    s.seller_state,
    ROUND(SUM(oi.price), 2) AS total_revenue,
    COUNT(*) AS total_order_lines
FROM sellers s
JOIN order_items oi
    ON s.seller_id = oi.seller_id
GROUP BY s.seller_id, s.seller_city, s.seller_state
ORDER BY total_revenue DESC
LIMIT 10;
```

Finding
Top sellers include:
seller 4869f7... from Guariba, SP: about 229,472.63
seller 532435... from Lauro de Freitas, BA: about 222,776.05
seller 4a3ca9... from Ibitinga, SP: about 200,472.92

Managerial importance
Helps Target identify strategic seller partners and manage seller quality programs.

```
218 •  SELECT
219        s.seller_id,
220        s.seller_city,
221        s.seller_state,
222        ROUND(SUM(oi.price), 2) AS total_revenue,
223        COUNT(*) AS total_order_lines
224    FROM sellers s
225    JOIN order_items oi
226        ON s.seller_id = oi.seller_id
227    GROUP BY s.seller_id, s.seller_city, s.seller_state
228    ORDER BY total_revenue DESC
229    LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

seller_id	seller_city	seller_state	total_revenue	total_order_lines
4869f7a5dfa277a7dca6462dcf3b52b2	guariba	SP	229472.63	1156
53243585a1d6dc2643021fd1853d8905	lauro de freitas	BA	222776.05	410
4a3ca9315b744ce9f8e9374361493884	ibitinga	SP	200472.92	1987
fa1c13f2614d7b5c4749cbc52fecda94	sumare	SP	194042.03	586
7c67e1448b00f6e969d365cea6b010ab	itaquaquecetuba	SP	187923.89	1364
7e93a43ef30c4f03f38b393420bc753a	barueri	SP	176431.87	340
da8622b14eb17ae2831f4ac5b9dab84a	piracicaba	SP	160236.57	1551
7a67c85e85bb2ce8582c35f2203ad736	sao paulo	SP	141745.53	1171
1025f0e2d44d7041d6cf58b6550e0bfa	sao paulo	SP	138968.55	1428
955fee9216a65b617aa5c0531780ce60	sao paulo	SP	135171.70	1499

Query 6 — Delivery delay rate
Purpose: Measure logistics performance.

```
SELECT
    COUNT(*) AS delivered_orders,
```

```
  SUM(CASE
        WHEN order_delivered_customer_date > order_estimated_delivery_date THEN 1
        ELSE 0
     END) AS late_orders,
  ROUND(
     100 * SUM(CASE
              WHEN order_delivered_customer_date > order_estimated_delivery_date THEN 1
              ELSE 0
            END) / COUNT(*), 2
  ) AS late_order_rate_pct,
  ROUND(AVG(CASE
            WHEN order_delivered_customer_date > order_estimated_delivery_date
            THEN DATEDIFF(order_delivered_customer_date, order_estimated_delivery_date)
          END), 2) AS avg_days_late
FROM orders
WHERE order_delivered_customer_date IS NOT NULL
  AND order_estimated_delivery_date IS NOT NULL;
```

Finding
Late delivery rate is about 6.77%
Late orders are on average about 10.62 days late

Managerial importance
This directly informs carrier performance monitoring and customer experience improvement.

```
231 • SELECT
232       COUNT(*) AS delivered_orders,
233       SUM(CASE
234               WHEN order_delivered_customer_date > order_estimated_delivery_date THEN 1
235               ELSE 0
236           END) AS late_orders,
237       ROUND(
238           100 * SUM(CASE
239                       WHEN order_delivered_customer_date > order_estimated_delivery_date THEN 1
240                       ELSE 0
241                     END) / COUNT(*), 2
242       ) AS late_order_rate_pct,
243       ROUND(AVG(CASE
244                   WHEN order_delivered_customer_date > order_estimated_delivery_date
245                   THEN DATEDIFF(order_delivered_customer_date, order_estimated_delivery_date)
246                 END), 2) AS avg_days_late
247   FROM orders
248   WHERE order_delivered_customer_date IS NOT NULL
249     AND order_estimated_delivery_date IS NOT NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

delivered_orders	late_orders	late_order_rate_pct	avg_days_late
99441	7827	7.87	8.87

Query 7 — Review score by delivery punctuality
Purpose: See whether logistics affects satisfaction.
Uses JOIN

```
SELECT
   CASE
      WHEN o.order_delivered_customer_date > o.order_estimated_delivery_date THEN 'Late'
      WHEN o.order_delivered_customer_date = o.order_estimated_delivery_date THEN 'On
time'
      ELSE 'Early'
   END AS delivery_status,
   ROUND(AVG(r.review_score), 2) AS avg_review_score,
   COUNT(*) AS review_count
FROM orders o
JOIN order_reviews r
   ON o.order_id = r.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
  AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY delivery_status
ORDER BY avg_review_score DESC;
```

Finding
Average review scores are roughly:
Early: 4.29
On time: 4.03
Late: 2.27

Managerial importance
This is one of the strongest business insights: poor delivery performance sharply reduces customer satisfaction.

```
251 • SELECT
252       CASE
253           WHEN o.order_delivered_customer_date > o.order_estimated_delivery_date THEN 'Late'
254           WHEN o.order_delivered_customer_date = o.order_estimated_delivery_date THEN 'On time'
255           ELSE 'Early'
256       END AS delivery_status,
257       ROUND(AVG(r.review_score), 2) AS avg_review_score,
258       COUNT(*) AS review_count
259   FROM orders o
260   JOIN order_reviews r
261       ON o.order_id = r.order_id
262   WHERE o.order_delivered_customer_date IS NOT NULL
263     AND o.order_estimated_delivery_date IS NOT NULL
264   GROUP BY delivery_status
265   ORDER BY avg_review_score DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

delivery_status	avg_review_score	review_count
Early	4.22	90777
Late	2.57	7633

Query 8 — Categories with below-average review score
Purpose: Find weak product areas.
Uses JOIN + SUBQUERY

```
SELECT
    p.`product category` AS product_category,
    ROUND(AVG(r.review_score), 2) AS avg_review_score,
    COUNT(*) AS review_count
FROM order_reviews r
JOIN orders o ON r.order_id = o.order_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.`product category`
HAVING AVG(r.review_score) < (
    SELECT AVG(review_score)
    FROM order_reviews
)
ORDER BY avg_review_score ASC
LIMIT 10;
```

Finding
Below-average categories include things like: Furniture office, Fashion Men’s Clothing, audio, House comfort

Managerial importance
Helps Target prioritize product quality review, supplier evaluation, and category-specific service improvement.

```
267 •  SELECT
268        p.`product category` AS product_category,
269        ROUND(AVG(r.review_score), 2) AS avg_review_score,
270        COUNT(*) AS review_count
271    FROM order_reviews r
272    JOIN orders o
273        ON r.order_id = o.order_id
274    JOIN order_items oi
275        ON o.order_id = oi.order_id
276    JOIN products p
277        ON oi.product_id = p.product_id
278    GROUP BY p.`product category`
279    HAVING AVG(r.review_score) < (
280        SELECT AVG(review_score)
281        FROM order_reviews
282    )
283    ORDER BY avg_review_score ASC
284    LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_category	avg_review_score	review_count
insurance and services	2.50	2
Hygiene diapers	3.26	39
Kitchen portable and food coach	3.27	15
PC Gamer	3.33	9
Furniture office	3.49	1677
House Comfort 2	3.63	27
Fashion Men's Clothing	3.64	130
fixed telephony	3.68	262
party articles	3.77	43
Fashion Women's Clothing	3.78	49

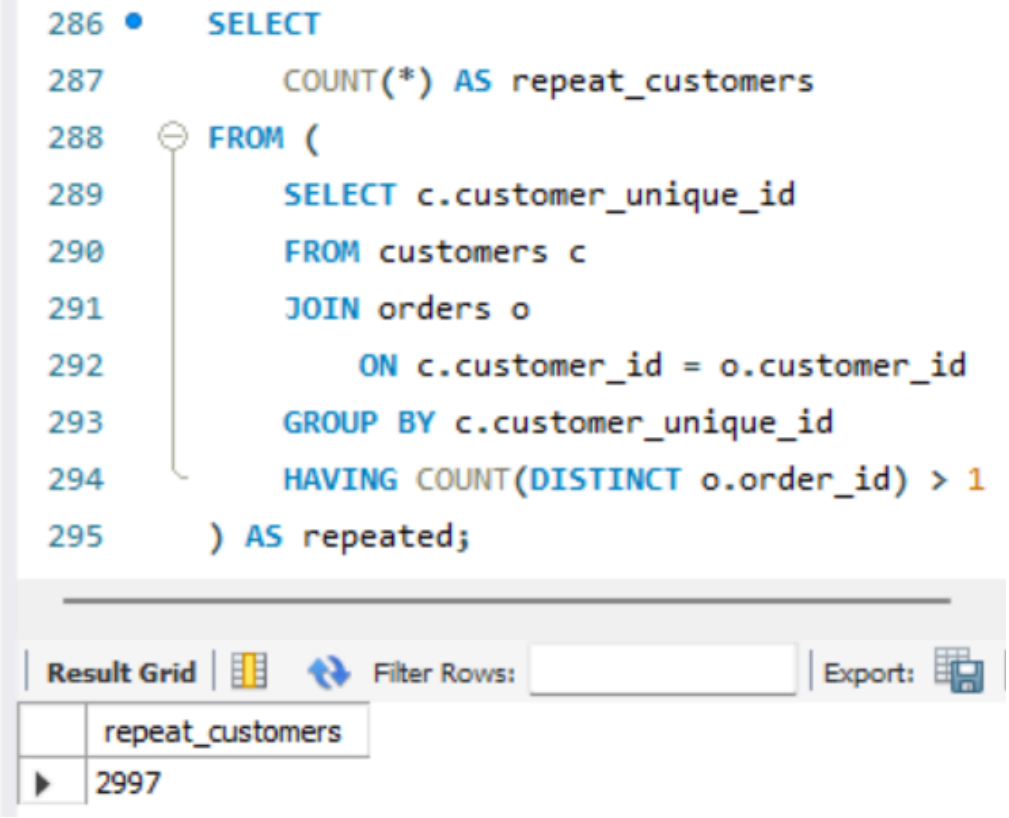
Query 9 — Repeat customers
Purpose: Measure loyalty.

Uses SUBQUERY

```
SELECT
   COUNT(*) AS repeat_customers
FROM (
   SELECT c.customer_unique_id
   FROM customers c
   JOIN orders o
      ON c.customer_id = o.customer_id
   GROUP BY c.customer_unique_id
   HAVING COUNT(DISTINCT o.order_id) > 1
) AS repeated;
```

Finding
There are about 2,997 repeat customers, which is only about 3.12% of unique customers.

Managerial importance
Shows customer retention is relatively low, so loyalty programs and post-purchase engagement could be valuable.



Query 10 — States with above-average order value
Purpose: Find premium markets.

Uses JOIN + SUBQUERY

```
SELECT
   c.customer_state,
   ROUND(AVG(oi.price), 2) AS avg_item_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_state
HAVING AVG(oi.price) > (
   SELECT AVG(price)
   FROM order_items
)
ORDER BY avg_item_value DESC;
```

Finding

Higher-value states include places such as:
PB
PA
MT
CE
MS

Managerial importance
Useful for regional pricing, premium assortment planning, and targeted marketing.

```
297 •  SELECT
298        c.customer_state,
299        ROUND(AVG(oi.price), 2) AS avg_item_value
300    FROM customers c
301    JOIN orders o
302        ON c.customer_id = o.customer_id
303    JOIN order_items oi
304        ON o.order_id = oi.order_id
305    GROUP BY c.customer_state
306    HAVING AVG(oi.price) > (
307        SELECT AVG(price)
308        FROM order_items
309    )
310    ORDER BY avg_item_value DESC;
```

Result Grid | Filter Rows: | Export: | Wrap

customer_state	avg_item_value
PB	191.48
AL	180.89
AC	173.73
RO	165.97
PA	165.69
AP	164.32
PI	160.36
TO	157.53
RN	156.97
CE	153.76
SE	153.04

Presentation Slides

- **This part is independent of your available data:**

1) Context and business background information (5 points).
   - Which company/organization will benefit from this database?
   - What entities or attributes might provide them with a competitive advantage over other companies, even if data on these entities or attributes is not available.
2) Metadata
   - describing at least five entities (5 points). (Description and examples)
   - describing at least five attributes for five entities (10 points). (Data value type, potential min, max, how the missing is handled, etc.)
3) ERD
   - Show the ER Diagram (No PK, FKs here). (10 points)
   - Describe all the relationships in text. (5 points)
   - Do you have any specific business rules that is reflected in the above relationships. (5 point)
4) Relational data model (10 points).
   - This is where you add PKs and FKs based on your ERD.

Presentation Slides (Continued)

- **This part is based on your available data:**

5) As-is dependency diagrams for your **actual** data tables (10 points).
6) If your **actual** data tables are not in 3rd normal form, convert them to 3rd normal form (10 points).
7) Screenshots of your **actual** data tables in 3rd normal form (5 points).
8) Screenshot of at least ten queries and the results. Using at least one JOINS (inner or outer) and one subquery is required (10 points).
9) For each query write/discuss the importance of each query/finding (10 points).
10) Presentation quality (visuals and style effort) (3 points).
11) Scope for extension. If you had more time and resources, how would you extend the current project? (2 points)

LINK TO SLIDES

https://www.canva.com/design/DAHDxrP_MTw/injQZ1wCwx8MrTAUu_beOg/edit?utm_content=DAHDxrP_MTw&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton